



UNIVERSITÉ
**PARIS
SCIENCES
& LETTRES**

Master Year 1 Mémoire

Advanced Mathematics Major

Market Making and Optimal Inventory Management

Authors

Marçal Herraiz Bayó
Carlos Cosentino Corzo
Tiziano Di Gregorio Botek

Supervisor: Dr. Philippe Bergault

Date: April 17, 2026

Contents

1	A Discrete-Time Model for Optimal Market Making	1
1.1	The Idea Behind Market Making	1
1.2	Discrete-Time Model of Market Making	2
1.3	Optimal Control Formulation and Solution	4
1.4	Numerical Experiments and Analysis of the Optimal Strategy	8
1.4.1	Validation and Profitability of the Optimal Strategy	8
1.4.2	Inventory-Dependent Quoting Behavior	9
1.4.3	Impact of Risk Aversion	10
1.4.4	Market Maker Efficient Frontier	11
2	A Continuous-Time Model for Market Making	12
2.1	Model Setup	12
2.2	Dynamic Programming Principle and HJB Equation	14
2.2.1	From DPP to HJB	15
2.3	Some Mild Assumptions	16
2.4	Existence and Uniqueness for the HJB equation	17
2.5	The Verification Theorem	18
2.6	Numerical Illustration of the Continuous-Time Optimal Strategy	20
2.6.1	Consistency Check and Profitability	21
2.6.2	Inventory Control Through Quotes	21
2.6.3	Risk-Return Trade-Off	23
A	Simulation Code	24
A.1	Discrete-Time Model	24
A.2	Continuous-Time Model	26
	Bibliography	31

Abstract

This thesis studies the problem of optimal market making under inventory risk, with the goal of balancing profitability from bid–ask spreads against the risk associated with holding positions. Building on the Avellaneda–Stoikov framework [1], we develop and analyze both a discrete-time and a continuous-time model for a market maker operating in a stochastic price environment.

In the discrete-time setting, the problem is formulated as a finite-horizon stochastic control problem, where the market maker dynamically chooses bid and ask quotes to maximize the expected profit penalized by inventory risk. Using dynamic programming, we derive the associated Bellman equation and characterize the optimal quoting strategy in terms of value function differences. Numerical simulations illustrate key features of the strategy, including inventory-dependent quoting behavior, the impact of risk aversion, and the resulting risk–return trade-off.

We then extend the model to continuous time, where asset prices follow a Brownian motion and order arrivals are modeled by controlled point processes. We consider a variation of Model B in [2], with different admissible controls and assumptions. The optimization problem is expressed through a Hamilton–Jacobi–Bellman equation, for which we establish existence and uniqueness of solutions under mild assumptions. A verification theorem confirms that the solution to the HJB equation indeed characterizes the value function and allows us to compute the optimal control policies. The resulting control policies retain the intuitive structure of the discrete-time case, with quotes acting as a feedback mechanism to manage inventory.

Finally, numerical experiments in both frameworks confirm the consistency of the theoretical results and demonstrate that optimal strategies generate positive expected profits while effectively controlling risk. The efficient frontier obtained by varying the risk-aversion parameter highlights the fundamental trade-off between return and risk faced by market makers.

Chapter 1

A Discrete-Time Model for Optimal Market Making

Market making is a central activity in modern financial markets, providing liquidity and enabling efficient trading. In this chapter, we introduce the concept of market making and highlight the fundamental trade-off between profitability and inventory risk faced by a market maker. We then develop a discrete-time mathematical model inspired by the Avellaneda–Stoikov framework [1], which allows us to formalize this trade-off as an optimal control problem. Finally, we study the resulting optimal strategy through numerical simulations,¹ illustrating its key features and the impact of model parameters on the market maker’s behavior.

1.1 The Idea Behind Market Making

A market maker is a financial intermediary whose primary role is to provide liquidity to a financial market. To achieve this, the market maker continuously posts two prices:

- Bid price: the price at which they are willing to buy the asset.
- Ask price: the price at which they are willing to sell the asset.

The difference between these two prices, known as the bid–ask spread, constitutes the main source of profit for the market maker (see Figure 1.1).

By standing ready to trade on both sides of the market, market makers facilitate transactions for other participants, ensuring that buyers and sellers can execute trades quickly and without having to wait for a counterparty.

Market making, however, involves a fundamental trade-off between execution probability and risk. Setting quotes far from the current market price reduces the likelihood of execution, thereby limiting trading opportunities and expected profits. Conversely, quoting prices close to the market price increases the probability of execution but exposes the market maker to inventory risk. This risk arises because

¹All the code developed during this work is publicly available in a GitHub repository [3].



Figure 1.1: Simulated asset price as a Brownian motion path, together with bid and ask quotes set around the asset price. The vertical separation between the bid and ask curves represents the bid–ask spread.

executed trades leave the market maker holding a position (long or short) whose value may evolve unfavorably as prices fluctuate.

As a result, market makers must carefully choose their quoted prices to balance two competing objectives: maximizing the expected profit from the bid–ask spread and controlling the risk associated with holding inventory. In practice, this requires dynamically adjusting quotes in response to market conditions such as price movements or volatility.

Example 1.1. Consider a stock currently trading at €100. A market maker may quote a bid price of €99.90 and an ask price of €100.10. If a trader sells to the market maker at €99.90 and another trader later buys from them at €100.10, the market maker earns €0.20 per share. However, if the price drops to €98 after they purchase the asset, they incur a loss due to the decline in the value of their inventory.

1.2 Discrete-Time Model of Market Making

To formalize the trade-off described in the previous section, we introduce a mathematical model of a market maker based on a discrete-time version of the Avellaneda-Stoikov model. Thus, we consider a market with a single traded asset in discrete time. At each time step $n \in \mathbb{N}$, the asset price S_n evolves according to the stochastic recursion

$$S_{n+1} = S_n + \xi_n,$$

where the increments $(\xi_n)_{n \in \mathbb{N}}$ are i.i.d. random variables such that

$$\xi_n \sim \mathcal{N}(0, \sigma^2).$$

At each time step, the market maker posts two quotes:

- an *ask price* $S_n^a = S_n + \delta_n^a$, at which they are willing to sell,
- a *bid price* $S_n^b = S_n - \delta_n^b$, at which they are willing to buy,

where $\delta_n^a, \delta_n^b \in \mathbb{R}_+$ are control variables representing the distance from the asset price.

On top of this, the market maker manages two accounts:

- a *cash account* $(X_n)_{n \in \mathbb{N}}$, where X_n represents the cumulative cash held by the market maker at time n . It evolves as trades are executed and is initialized as $X_0 = 0$,
- an *inventory process* $(q_n)_{n \in \mathbb{N}}$, where q_n denotes the number of units of the asset held by the market maker at time n (positive for a long position, negative for a short position), with initial condition $q_0 = 0$.

At each time step n , at most one transaction can occur: either a buy, a sell, or no trade. To model this, we introduce two Bernoulli random variables η_n^a and η_n^b , defined as follows:

- $\eta_n^a = 1$ if the market maker sells one unit of the asset (i.e., a trade occurs at the ask price),
- $\eta_n^b = 1$ if the market maker buys one unit of the asset (i.e., a trade occurs at the bid price).

Assumption 1.2. *At each time step, trades cannot occur simultaneously on both sides of the market. That is, η_n^a and η_n^b cannot both be equal to 1.*

As explained in Section 1.1, execution probabilities decrease as the quotes move away from the asset price. Accordingly, let $f: \mathbb{R}_+ \rightarrow [0, 1/2]$ be a decreasing measurable function. We model these probabilities as

$$\begin{aligned} \mathbb{P}(\text{market maker buys at bid}) &= \mathbb{P}(\eta_n^b = 1, \eta_n^a = 0) = f(\delta_n^b), \\ \mathbb{P}(\text{market maker sells at ask}) &= \mathbb{P}(\eta_n^b = 0, \eta_n^a = 1) = f(\delta_n^a), \end{aligned}$$

and the probability that no trade occurs is given by

$$\mathbb{P}(\eta_n^b = 0, \eta_n^a = 0) = 1 - f(\delta_n^a) - f(\delta_n^b).$$

Having determined the conditional distribution (with respect to (δ_n^a, δ_n^b)) of the random vector (η_n^a, η_n^b) , we further assume that it is independent from $(\xi_m)_{m \geq n}$. This is indeed a natural assumption: whether we buy or sell a stock (or neither) at time n cannot depend on the future increments of the stock price.

Since the inventory increases (resp. decreases) by one unit when the market maker buys (resp. sells) at the bid (resp. ask) price, its dynamics are given by

$$q_{n+1} = q_n + \eta_n^b - \eta_n^a.$$

It is easily shown by induction that q_{n+1} is independent of ξ_n for all $n \in \mathbb{N}$.

Similarly, each transaction updates the cash account. Selling at the ask price adds $S_n + \delta_n^a$ to the market maker's cash account, whereas buying at the bid price decreases it by $S_n - \delta_n^b$. Thus, the cash process evolves as

$$X_{n+1} = X_n + \eta_n^a(S_n + \delta_n^a) - \eta_n^b(S_n - \delta_n^b).$$

With this, the total value of the market maker's portfolio at any given time $N \in \mathbb{N}$ is given by the sum of the cash account, X_N , and the inventory multiplied by the stock price, $q_N S_N$. A direct computation then shows that

$$X_N + q_N S_N = \sum_{n=0}^{N-1} \left(\eta_n^a \delta_n^a + \eta_n^b \delta_n^b + q_{n+1} \xi_n \right). \quad (1.1)$$

When we take the expectation on the above expression, the last term in the sum vanishes, since

$$\mathbb{E}[q_{n+1} \xi_n] = \mathbb{E}[q_{n+1}] \cdot \mathbb{E}[\xi_n] = 0,$$

where we used independence and the fact that ξ_n is centered.

To account for inventory risk, we introduce a risk-aversion parameter $\gamma > 0$ and consider, for a fixed time horizon $N \in \mathbb{N}$, the following objective function:

$$\mathbb{E} \left[X_N + q_N S_N - \frac{\gamma}{2} \sum_{n=0}^{N-1} q_n^2 \right].$$

Using (1.1), this can be rewritten as

$$\mathbb{E} \left[\sum_{n=0}^{N-1} \left(\eta_n^a \delta_n^a + \eta_n^b \delta_n^b - \frac{\gamma}{2} q_n^2 \right) \right].$$

The market maker's problem is therefore to choose the control variables (δ_n^a, δ_n^b) at each time step in order to maximize this objective. That is, to solve

$$\sup_{\{(\delta_n^a, \delta_n^b)\}_{n=0}^{N-1}} \mathbb{E} \left[X_N + q_N S_N - \frac{\gamma}{2} \sum_{n=0}^{N-1} q_n^2 \right]. \quad (1.2)$$

1.3 Optimal Control Formulation and Solution

We now solve the market maker's optimization problem.

Assumption 1.3. *The function $f: \mathbb{R}_+ \rightarrow [0, 1/2]$ is continuous, decreasing, and satisfies the following fast-decay condition:*

$$\lim_{\lambda \rightarrow +\infty} \lambda f(\lambda) = 0. \quad (1.3)$$

Recall that the inventory process $(q_n)_{n=0}^N$ evolves as

$$q_0 = 0, \quad q_{n+1} = q_n + \eta_n^b - \eta_n^a,$$

where, at each time $n \in \mathbb{N}$, we choose the control variables $(\delta_n^a, \delta_n^b) \in \mathbb{R}_+^2$, which in turn determine the joint conditional law of (η_n^a, η_n^b) independently of the past through

$$\mathbb{P}(\eta_n^a = i, \eta_n^b = j) = \begin{cases} f(\delta_n^a), & i = 1, j = 0, \\ f(\delta_n^b), & i = 0, j = 1, \\ 1 - f(\delta_n^a) - f(\delta_n^b), & i = j = 0. \end{cases}$$

Thus, $(q_n)_{n \in \mathbb{N}}$ is a controlled random walk on $\mathbb{Z}$ starting from 0, with time-homogeneous transition probabilities

$$P(x, y, (\delta^a, \delta^b)) = \begin{cases} f(\delta^a), & y = x - 1, \\ f(\delta^b), & y = x + 1, \\ 1 - f(\delta^a) - f(\delta^b), & y = x, \\ 0, & \text{otherwise.} \end{cases}$$

We now claim that the cash account $X = (X_n)_{n \in \mathbb{N}}$, the Bernoulli random variables $((\eta_n^a, \eta_n^b))_{n \in \mathbb{N}}$ and the stock price $S = (S_n)_{n \in \mathbb{N}}$ (including the distribution of the random variables $(\xi_n)_{n \in \mathbb{N}}$ as long as they are centered) are irrelevant to our problem. Indeed, we aim to find, for some fixed time horizon $N \in \mathbb{N}$,

$$\sup_{\{(\delta_n^a, \delta_n^b)\}_{n=0}^{N-1}} \mathbb{E} \left[\sum_{n=0}^{N-1} \left(\eta_n^a \delta_n^a + \eta_n^b \delta_n^b - \frac{\gamma}{2} q_n^2 \right) \right].$$

Applying an intermediate conditional expectation, we get

$$\mathbb{E}[\eta_n^a \delta_n^a] = \mathbb{E}[\delta_n^a \mathbb{E}[\eta_n^a \mid \delta_n^a, \delta_n^b]] = \mathbb{E}[\delta_n^a f(\delta_n^a)],$$

and the same holds for the bid side.

Defining the gain function $c: \mathbb{N} \times \mathbb{Z} \times \mathbb{R}_+^2 \rightarrow \mathbb{R}$ by

$$c(n, q, (\delta^a, \delta^b)) = \begin{cases} \delta^a f(\delta^a) + \delta^b f(\delta^b) - \frac{\gamma}{2} q^2, & n \in \{0, \dots, N-1\}, \\ 0, & n \geq N, \end{cases}$$

our optimal control problem becomes

$$V_{\text{opt}} = \sup_{\{(\delta_n^a, \delta_n^b)\}_{n=0}^N} \mathbb{E} \left[\sum_{n=0}^N c(n, q_n, (\delta_n^a, \delta_n^b)) \right].$$

Notice this problem only involves the controlled random walk $(q_n)_{n=0}^N$ and its control variables $(\delta_n^a, \delta_n^b)_{n=0}^N$. To solve this finite-horizon problem, we make use of the associated value function $V: \mathbb{N} \times \mathbb{Z} \rightarrow \mathbb{R}$, defined as

$$V(k, q) := \sup_{\{(\delta_n^a, \delta_n^b)\}_{n=k}^{N-1}} \mathbb{E} \left[\sum_{n=k}^N c(n, q_n, (\delta_n^a, \delta_n^b)) \right].$$

As $V(N, q) = 0$ for all $q \in \mathbb{Z}$, we may apply backward induction using the Bellman equation. For this purpose, it is convenient to introduce the following maximization problems for any $\lambda \in \mathbb{R}$:

$$H(\lambda) := \max_{\delta \geq 0} \{(\delta - \lambda)f(\delta)\}, \quad \Delta(\lambda) := \arg \max_{\delta \geq 0} \{(\delta - \lambda)f(\delta)\}. \quad (1.4)$$

The maxima are indeed attained thanks to the continuity of f and Assumption 1.3, so $\Delta(\lambda) \subset \mathbb{R}_+$ is non-empty for all $\lambda \in \mathbb{R}$. In the following, let $Z \sim P(q, \cdot, (\delta^a, \delta^b))$. Developing, we have:

$$V(N-1, q) = \sup_{\delta^a, \delta^b \geq 0} \left\{ c(N-1, q, (\delta^a, \delta^b)) + PV(N-1, q, (\delta^a, \delta^b)) \right\}. \quad (1.5)$$

Note also that

$$PV(N-1, q, (\delta^a, \delta^b)) := \mathbb{E}[V(N, Z)] = 0. \quad (1.6)$$

Therefore, using (1.5) and (1.6) yields

$$\begin{aligned} V(N-1, q) &= \sup_{\delta^a, \delta^b \geq 0} \left\{ c(N-1, q, (\delta^a, \delta^b)) \right\} \\ &= \sup_{\delta^a, \delta^b \geq 0} \left\{ \delta^a f(\delta^a) + \delta^b f(\delta^b) - \frac{\gamma}{2} q^2 \right\} \\ &= 2C(0) - \frac{\gamma}{2} q^2, \end{aligned} \quad (1.7)$$

and the control is optimal if and only if:

$$(\delta_{N-1}^a, \delta_{N-1}^b) \in \Delta(0) \times \Delta(0).$$

Next, we proceed one step further:

$$V(N-2, q) = \sup_{\delta^a, \delta^b \geq 0} \left\{ c(N-2, q, (\delta^a, \delta^b)) + PV(N-2, q, (\delta^a, \delta^b)) \right\}. \quad (1.8)$$

We need the following expectation:

$$\begin{aligned} \mathbb{E}[Z^2] &= f(\delta^a)(q-1)^2 + f(\delta^b)(q+1)^2 + (1-f(\delta^a)-f(\delta^b))q^2 \\ &= q^2 + f(\delta^a)(1-2q) + f(\delta^b)(1+2q). \end{aligned}$$

With this, we have

$$\begin{aligned} PV(N-2, q, (\delta^a, \delta^b)) &:= \mathbb{E}[V(N-1, Z)] \\ &= 2C(0) - \frac{\gamma}{2} \mathbb{E}[Z^2] \\ &= 2C(0) - \frac{\gamma}{2} \left(q^2 + f(\delta^a)(1-2q) + f(\delta^b)(1+2q) \right). \end{aligned} \quad (1.9)$$

Substituting (1.9) in (1.8), we obtain

$$\begin{aligned} V(N-2, q) &= 2H(0) - \gamma q^2 \\ &\quad + \sup_{\delta^a, \delta^b \geq 0} \left\{ \left(\delta^a - \frac{\gamma}{2}(1-2q) \right) f(\delta^a) + \left(\delta^b - \frac{\gamma}{2}(1+2q) \right) f(\delta^b) \right\} \\ &= 2H(0) + H\left(\frac{\gamma}{2}(1-2q)\right) + H\left(\frac{\gamma}{2}(1+2q)\right) - \gamma q^2, \end{aligned} \quad (1.10)$$

and a control is optimal if and only if

$$(\delta_{N-2}^a, \delta_{N-2}^b) \in \Delta\left(\frac{\gamma}{2}(1-2q)\right) \times \Delta\left(\frac{\gamma}{2}(1+2q)\right).$$

Beyond this point, computing explicit formulas for $V(N-3, q)$ and in general $V(N-k, q)$ becomes cumbersome. Instead, it is more convenient to use a backward-induction algorithm to calculate the value function.

Theorem 1.4. *The value function $V: \{0, \dots, N\} \times \mathbb{Z} \rightarrow \mathbb{R}$ is the unique solution to the following system:*

$$\begin{cases} V(N, q) = 0, \\ V(n, q) = V(n+1, q) - \frac{\gamma}{2}q^2 \\ \quad + H(V(n+1, q) - V(n+1, q-1)) \\ \quad + H(V(n+1, q) - V(n+1, q+1)). \end{cases}$$

Moreover, the optimal controls at time $n = 0, \dots, N-1$ satisfy:

$$\begin{aligned} \hat{\delta}_n^a &\in \Delta(V(n+1, q_n) - V(n+1, q_n-1)), \\ \hat{\delta}_n^b &\in \Delta(V(n+1, q_n) - V(n+1, q_n+1)). \end{aligned}$$

Proof. The terminal value has already been shown to be zero. Fix $n \in \{0, \dots, N-1\}$. Then we have the following:

$$\begin{aligned} PV(n, q, (\delta^a, \delta^b)) &= \mathbb{E}[V(n+1, Z)] \\ &= f(\delta^a)V(n+1, q-1) + f(\delta^b)V(n+1, q+1) \\ &\quad + (1 - f(\delta^a) - f(\delta^b))V(n+1, q) \\ &= V(n+1, q) + f(\delta^a)(V(n+1, q-1) - V(n+1, q)) \\ &\quad + f(\delta^b)(V(n+1, q+1) - V(n+1, q)). \end{aligned}$$

Using Bellman's Equation, we find:

$$\begin{aligned} V(n, q) &= \sup_{\delta^a, \delta^b \geq 0} \{ \delta^a f(\delta^a) + \delta^b f(\delta^b) - \frac{\gamma}{2}q^2 + PV(n, q, (\delta^a, \delta^b)) \} \\ &= V(n+1, q) - \frac{\gamma}{2}q^2 + \sup_{\delta^a \geq 0} \left\{ (\delta^a - (V(n+1, q) - V(n+1, q-1)))f(\delta^a) \right\} \\ &\quad + \sup_{\delta^b \geq 0} \left\{ (\delta^b - (V(n+1, q) - V(n+1, q+1)))f(\delta^b) \right\} \\ &= V(n+1, q) - \frac{\gamma}{2}q^2 + H(V(n+1, q) - V(n+1, q-1)) \\ &\quad + H(V(n+1, q) - V(n+1, q+1)), \end{aligned}$$

which proves the first part of the theorem, where uniqueness follows immediately by backward induction from the terminal condition $V(N, q) = 0$.

For the second part, note that by the definition of the set-valued map Δ , the suprema are attained if and only if

$$(\delta^a, \delta^b) \in \Delta(V(n+1, q) - V(n+1, q-1)) \times \Delta(V(n+1, q) - V(n+1, q+1)),$$

as was to be shown. $\square$

Remark 1.5. By the axiom of choice, there exists a map $F: \mathbb{R} \rightarrow \mathbb{R}$ such that $F(\lambda) \in \Delta(\lambda)$ for all $\lambda \in \mathbb{R}$. Then we can form a Markovian admissible optimal control by setting

$$\begin{aligned}\hat{\delta}_n^a &:= F(V(n+1, q_n) - V(n+1, q_n - 1)), \\ \hat{\delta}_n^b &:= F(V(n+1, q_n) - V(n+1, q_n + 1)).\end{aligned}$$

Note that these are compositions of q_n with maps $\mathbb{Z} \rightarrow \mathbb{R}$, which are always measurable.

Remark 1.6. To compute $V(0, 0)$, one performs a backward induction starting from $V(N, q) = 0$ for all $q \in \{-N, \dots, N\}$, and then iteratively computes at each step n the value of $V(n, q)$ for $q \in \{-n, \dots, 0, \dots, n\}$. If N is large enough, it may be convenient to start the backward induction at time $N - 1$ or $N - 2$ using the explicit formulas (1.7) and (1.10). Once these values of $V(n, q)$ have been stored, the optimal controls can be implemented.

1.4 Numerical Experiments and Analysis of the Optimal Strategy

In this section, we illustrate the properties of the optimal strategy through numerical simulations. We consider the specific intensity function

$$f(x) = \frac{1}{2}e^{-x},$$

which satisfies Assumption 1.3. For this choice of f , the optimization problems in (1.4) defining H and Δ admit explicit solutions. Indeed, one easily obtains

$$H(\lambda) := \max_{\delta \geq 0} \{(\delta - \lambda)f(\delta)\} = \begin{cases} \frac{1}{2}e^{-(\lambda+1)}, & \text{if } \lambda \geq -1, \\ -\frac{\lambda}{2}, & \text{if } \lambda < -1, \end{cases}$$

and the corresponding optimizer set (a singleton in this case)

$$\Delta(\lambda) := \operatorname{argmax}_{\delta \geq 0} \{(\delta - \lambda)f(\delta)\} = \{(\lambda + 1)^+\}.$$

These explicit expressions allow us to compute the optimal bid and ask quotes throughout the trading horizon.

1.4.1 Validation and Profitability of the Optimal Strategy

To validate the numerical implementation, we estimate the expected value of the objective function using Monte Carlo simulations. Figure 1.2 shows the convergence of the empirical average toward the theoretical value $V(0, 0)$.

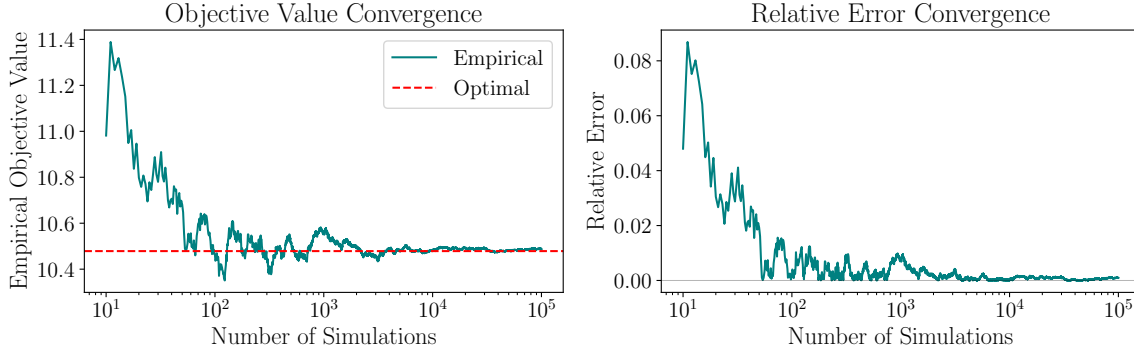


Figure 1.2: Monte Carlo convergence of the estimated value for the optimal strategy. The left panel presents the cumulative average of simulated costs compared to the optimal value $V(0, 0)$. The right panel shows the associated relative error as a percentage of this optimal value. Time horizon: 30.

We now examine the distribution of the terminal profit and loss (PnL) generated by the optimal strategy. For a given time horizon $N \in \mathbb{N}$, the terminal PnL is defined as the value of the portfolio at time N , namely

$$\text{PnL}_N := X_N + q_N S_N.$$

In other words, the terminal PnL is obtained by adding the cash accumulated through past trades to the market value of the inventory held at the end of the trading horizon.

As shown in Figure 1.3, the distribution is approximately bell-shaped and centered around a positive mean. This confirms that, on average, the market maker extracts profit from the bid–ask spread when optimally managing inventory risk.

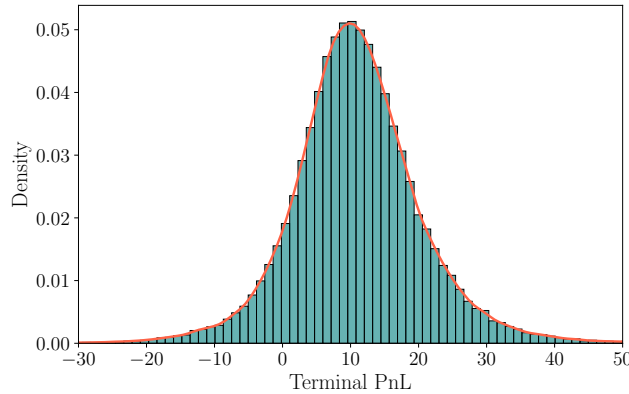


Figure 1.3: Distribution of the terminal PnL for the optimal strategy over $T = 30$ for 10^5 simulations. The kernel density estimate (red curve) is approximately bell-shaped with a positive mean.

1.4.2 Inventory-Dependent Quoting Behavior

Figure 1.4 illustrates a typical trajectory of the asset price together with the corresponding optimal bid and ask quotes at each time step, as well as the evolution of the inventory.

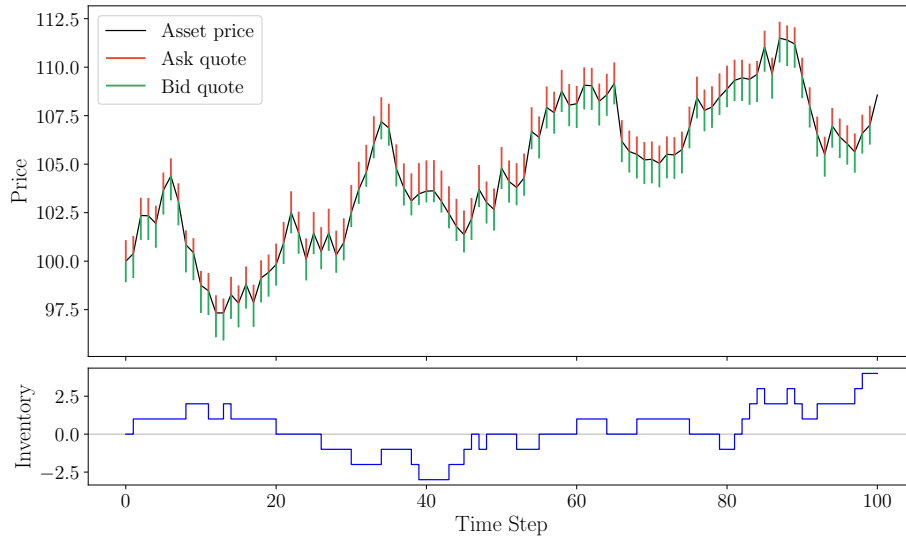


Figure 1.4: Optimal bid and ask quotes displayed around the asset price trajectory, together with the corresponding inventory evolution.

As expected, the quoting strategy adapts dynamically to the market maker's inventory position. When their position becomes long, i.e., $q > 0$, they become more aggressive on the ask side (lower δ^a) and more conservative on the bid side (higher δ^b), reflecting an incentive to reduce inventory. The opposite behavior occurs when the inventory is negative.

This dependence is further illustrated in Figure 1.5, which shows the optimal quotes as functions of the inventory at a fixed time.

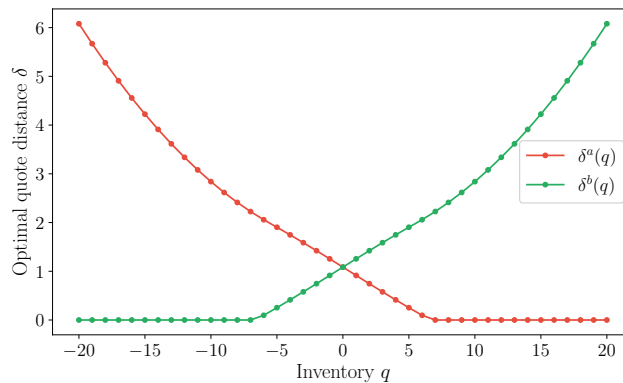


Figure 1.5: Optimal quotes as a function of the inventory for a fixed time $n = 30$ and a time horizon $T = 100$.

1.4.3 Impact of Risk Aversion

We now investigate the effect of the risk-aversion parameter γ . As γ increases, the penalty on inventory becomes stronger, leading to more conservative strategies and reduced exposure to price fluctuations.

This effect is illustrated in Figure 1.6, where the variance of the terminal PnL decreases as γ increases. The variance is normalized by $\text{Var}(S_T - S_0) = T\sigma^2$, allowing

for comparison with the intrinsic volatility of the asset. Thus, values below 1 mean that the terminal PnL variance is smaller than the variance of the underlying asset over the same horizon

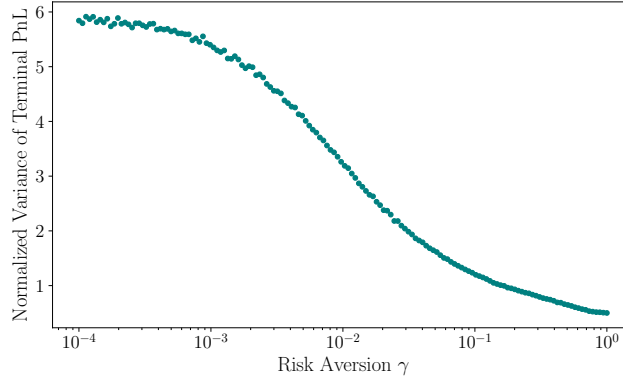


Figure 1.6: Normalized variance of the terminal PnL over $T = 30$ as a function of the risk-aversion parameter $\gamma \in [10^{-4}, 1]$.

1.4.4 Market Maker Efficient Frontier

Finally, following the technique used in [4], we analyze the trade-off between risk and return induced by different levels of risk aversion. Figure 1.7 displays the efficient frontier of the market maker, representing the expected terminal PnL as a function of its standard deviation (risk). These results have been obtained by Monte Carlo simulations of 10^5 trajectories for each of the 150 values of $\gamma \in [10^{-4}, 1]$.

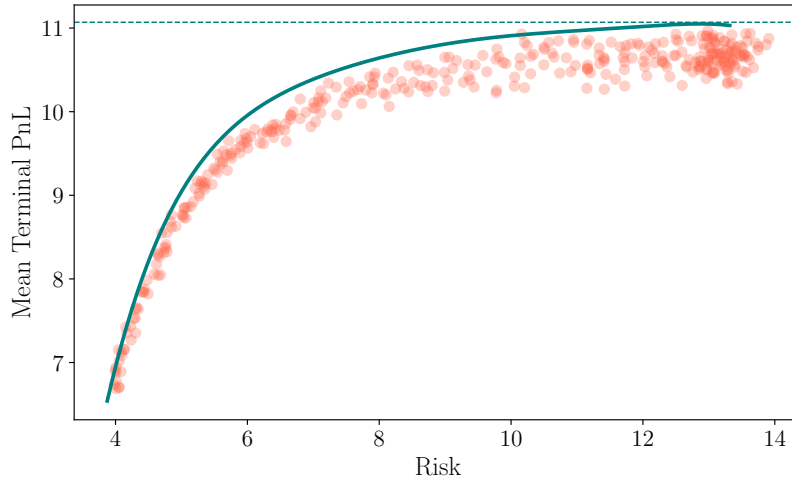


Figure 1.7: Efficient frontier of the market maker: expected terminal PnL as a function of risk for a time horizon $T = 30$. The frontier (solid line) is obtained via a polynomial fit to simulation results of the optimal strategy for different risk aversion levels, while red points show perturbed strategies. The dashed line indicates the expected PnL in the absence of inventory risk penalization.

The efficient frontier highlights the fundamental trade-off faced by the market maker: higher expected profits can be achieved at the cost of increased risk. The optimal strategies lie on the frontier, while suboptimal strategies fall below it.

Chapter 2

A Continuous-Time Model for Market Making

In this chapter, we adapt the previous model to continuous time. We consider a variation of Model B in [2], with different admissible controls and assumptions. As we will see, the solution is essentially the same in the continuous-time setting, at the cost of much more technical care.

2.1 Model Setup

Fix once and for all a terminal time $T > 0$ and a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \geq 0}, \mathbb{P})$ satisfying the standard conditions and supporting a standard $(\mathcal{F}_t)_{t \geq 0}$ -Brownian motion $W = (W_t)_{t \geq 0}$ and two independent $\mathcal{F}_0$ -measurable standard Poisson Processes N^1 and N^2 with unit intensity. We consider a single stock whose price process $(S_t)_{t \in [0, T]}$ evolves according to

$$dS_t = \sigma dW_t,$$

where $\sigma > 0$ is the volatility.

At each time $t \in [0, T]$, the market maker posts:

- the *ask price* $S_t^a = S_t + \delta_t^a$,
- the *bid price* $S_t^b = S_t - \delta_t^b$,

where $\delta_t^a, \delta_t^b \in \mathbb{R}_+$ are control variables representing the distance from the reference price S_t .

Definition 2.1. *For any $t \in [0, T]$, the controls $\delta^a = (\delta_s^a)_{s \in [t, T]}$ and $\delta^b = (\delta_s^b)_{s \in [t, T]}$ are said to be admissible if they are non-negative, bounded, and predictable¹ stochastic processes.*

¹Recall that the predictable σ -algebra on $\mathbb{R}_+ \times \Omega$ is the one generated by the adapted left-continuous processes. In particular, any adapted left-continuous process is predictable

Let $\Lambda^a, \Lambda^b: \mathbb{R}_+ \rightarrow \mathbb{R}_+$ be continuous, decreasing functions satisfying the fast-decay condition (1.3). The instantaneous rate at which a trades occur depends on (δ_t^a, δ_t^b) through the functions Λ^a, Λ^b . However, in this model we further impose an inventory constraint. Namely, that there is a quantity $Q \in \mathbb{N}$ such that the inventory process $q = (q_t)_{t \in [0, T]}$ takes values in $\{-Q, \dots, +Q\}$. This is enforced by allowing sales only when $q_{t-} > -Q$ and purchases only when $q_{t-} < Q$.

To model the arrivals of executed trades, we introduce two point processes N^a and N^b with respective time-dependent (random) intensities

$$\lambda_t^a = \mathbb{1}_{\{q_{t-} > -Q\}} \Lambda^a(\delta_t^a), \quad \lambda_t^b = \mathbb{1}_{\{q_{t-} < Q\}} \Lambda^b(\delta_t^b).$$

At each time $t \in [0, T]$, N_t^a counts the number of units sold by the market maker up to time t , while N_t^b counts the number of units bought up to time t .

Therefore, the inventory process q satisfies

$$dq_t = dN_t^b - dN_t^a, \quad q_0 \in \{-Q, \dots, Q\}.$$

Remark 2.2. Two adapted point processes N^a and N^b with given predictable random intensities can be constructed within our probability space using N^1, N^2 and suitable random time changes. However, here the intensities of N^a and N^b depend also on $(q_t)_{t \geq 0}$, which solves $dq_t = dN_t^b - dN_t^a$, so this is not as straightforward. The evolution of the triple (N^a, N^b, q) depends on itself at each point in time. The model described for the inventory process is thus a Stochastic Differential Equation of pure-jump type ('Jump SDE' for short). Existence and Uniqueness follow from the general theory of Jump SDEs.²

Note that now the inventory process is a continuous-time controlled random walk on $\{-Q, \dots, Q\}$. Similarly, the cash account $X = (X_t)_{t \in [0, T]}$ evolves according to

$$\begin{aligned} dX_t &= S_t^a dN_t^a - S_t^b dN_t^b \\ &= (S_t + \delta_t^a) dN_t^a - (S_t - \delta_t^b) dN_t^b \\ &= \delta_t^a dN_t^a + \delta_t^b dN_t^b - S_t dq_t, \end{aligned} \tag{2.1}$$

with initial condition $X_0 = 0$.

Thus, the total value of the market maker's portfolio is given by the process $X_t + q_t S_t$. Applying the product rule for the second term gives

$$d(q_t S_t) = S_t dq_t + q_t dS_t + dq_t dS_t = S_t dq_t + q_t dS_t,$$

where we used the fact that $dN_t^a dW_t = dN_t^b dW_t = 0$ to obtain that

$$dq_t dS_t = \sigma(dN_t^a W_t - dN_t^b dW_t) = 0.$$

Combining this with (2.1), we obtain

$$\begin{aligned} d(X_t + q_t S_t) &= \delta_t^a dN_t^a + \delta_t^b dN_t^b + q_t dS_t \\ &= \delta_t^a \lambda_t^a dt + \delta_t^b \lambda_t^b dt + \delta_t^a dM_t^a + \delta_t^b dM_t^b + \sigma q_t dW_t, \end{aligned}$$

²The Jump SDE may be phrased in the language of Poisson Random Measures, for which there exist very general well-posedness results. See [5], Chapter IV.

where M^a and M^b are the martingales

$$M_t^a = N_t^a - \int_0^t \lambda_s^a ds, \quad M_t^b = N_t^b - \int_0^t \lambda_s^b ds.$$

Integrating from 0 to t , we get

$$X_t + q_t S_t = \int_0^t (\delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b) ds + \int_0^t \delta_s^a dM_s^a + \int_0^t \delta_s^b dM_s^b + \sigma \int_0^t q_s dW_s.$$

Since q , δ^a , and δ^b are bounded, their stochastic integrals with respect to martingales are martingales. Therefore,

$$X_t + q_t S_t = \int_0^t (\delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b) ds + (\text{martingale starting at } 0),$$

and this implies that

$$\mathbb{E}[X_t + q_t S_t] = \mathbb{E} \left[\int_0^t (\delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b) ds \right]. \quad (2.2)$$

We now introduce a risk-aversion parameter $\gamma > 0$. The objective is to maximize the expected terminal value of the portfolio while penalizing large inventory positions over time. A natural generalization of the discrete-time objective function (1.2) is:

$$\mathbb{E} \left[X_T + q_T S_T - \frac{\gamma}{2} \sigma^2 \int_0^T q_s^2 ds \right].$$

Using the previous identity (2.2), the optimal control problem can be rewritten as

$$\sup_{(\delta_t^a, \delta_t^b)_{t \in [0, T]}} \mathbb{E} \left[\int_0^T \left(\delta_t^a \lambda_t^a + \delta_t^b \lambda_t^b - \frac{\gamma}{2} \sigma^2 q_t^2 \right) dt \right], \quad (2.3)$$

where the supremum is over all admissible controls $(\delta_t^a, \delta_t^b)_{t \in [0, T]}$ starting at 0.

2.2 Dynamic Programming Principle and HJB Equation

The method of solution relies on the very well-known Dynamic Programming Principle (DPP), whose proof is essentially the same as in the discrete-time case and thus we omit. Notice that the expression (2.3) we aim to maximize depends only on the random walk q and the control processes δ^a, δ^b . The initial stock price S_0 , for example, becomes irrelevant. For this reason, we define the value function $v : [0, T] \times \{-Q, \dots, +Q\}$ as

$$v(t, q) = \sup_{(\delta_s^a, \delta_s^b)_{s \in [t, T]}} \mathbb{E}^{t, q} \left[\int_t^T \left(\delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b - \frac{\gamma}{2} \sigma^2 q_s^2 \right) ds \right]$$

for $(\delta_s^a, \delta_s^b)_{s \in [t, T]}$ among all admissible controls starting at t . Here, $\mathbb{E}^{t, q}$ denotes expectation with respect to the conditioned probability measure $\mathbb{P}^{t, q} = \mathbb{P}(\cdot \mid q_t = q)$. That is, we basically start the model at time t with initial stock $q_t = q$. Note that, at this point, the value function can even fail to be measurable. The only thing that is clear is the terminal condition $v(T, q) = 0$ for all $q \in \{-Q, \dots, +Q\}$.

2.2.1 From DPP to HJB

In this subsection we derive heuristically why the value function is expected to solve a certain ODE terminal value problem, called the Hamilton–Jacobi–Bellman (HJB) equation.

The Dynamic Programming Principle (DPP) states that for every $h \geq 0$ such that $0 \leq t \leq t+h \leq T$,

$$v(t, q) = \sup_{(\delta_s^a, \delta_s^b)_{s \in [t, T]}} \mathbb{E}^{t, q} \left[v(t+h, q_{t+h}) + \int_t^{t+h} \left(\delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b - \frac{\gamma}{2} \sigma^2 q_s^2 \right) ds \right]. \quad (2.4)$$

Assume that the value function v , which can be seen as

$$v: [0, T] \longrightarrow \mathbb{R}^{2Q+1}, \quad t \longmapsto (v(t, q))_{q \in \{-Q, \dots, Q\}},$$

is of class C^1 . Applying Itô's formula for the pure-jump process $(q_s)_{s \in [t, T]}$ gives

$$\begin{aligned} v(t+h, q_{t+h}) &= v(t, q) + \int_t^{t+h} v'(s, q_s) ds \\ &\quad + \int_t^{t+h} (v(s, q_s + 1) - v(s, q_s)) dN_s^b \\ &\quad + \int_t^{t+h} (v(s, q_s - 1) - v(s, q_s)) dN_s^a. \end{aligned}$$

Since

$$dN_s^a = \lambda_s^a ds + dM_s^a, \quad dN_s^b = \lambda_s^b ds + dM_s^b,$$

and v is assumed to be bounded (it is C^1 on a compact interval), we have that

$$\begin{aligned} \mathbb{E}^{t, q} \left[\int_t^{t+h} (v(s, q_s - 1) - v(s, q_s)) dN_s^a \right] &= \mathbb{E}^{t, q} \left[\int_t^{t+h} (v(s, q_s - 1) - v(s, q_s)) \lambda_s^a ds \right], \\ \mathbb{E}^{t, q} \left[\int_t^{t+h} (v(s, q_s + 1) - v(s, q_s)) dN_s^b \right] &= \mathbb{E}^{t, q} \left[\int_t^{t+h} (v(s, q_s + 1) - v(s, q_s)) \lambda_s^b ds \right]. \end{aligned}$$

Hence,

$$\begin{aligned} &\mathbb{E}^{t, q} \left[v(t+h, q_{t+h}) + \int_t^{t+h} \left(\delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b - \frac{\gamma}{2} \sigma^2 q_s^2 \right) ds \right] \\ &= v(t, q) + \mathbb{E}^{t, q} \left[\int_t^{t+h} \left\{ v'(s, q_s) + \lambda_s^a (\delta_s^a - [v(s, q_s) - v(s, q_s - 1)]) \right. \right. \\ &\quad \left. \left. + \lambda_s^b (\delta_s^b - [v(s, q_s) - v(s, q_s + 1)]) - \frac{\gamma}{2} \sigma^2 q_s^2 \right\} ds \right]. \end{aligned}$$

Substituting this into (2.4), we obtain

$$\begin{aligned} 0 &= \sup_{(\delta_s^a, \delta_s^b)_{s \in [t, T]}} \mathbb{E}^{t, q} \left[\int_t^{t+h} \left\{ v'(s, q_s) + \mathbb{1}_{\{q_s > -Q\}} \Lambda^a(\delta_s^a) (\delta_s^a - [v(s, q_s) - v(s, q_s - 1)]) \right. \right. \\ &\quad \left. \left. + \mathbb{1}_{\{q_s < Q\}} \Lambda^b(\delta_s^b) (\delta_s^b - [v(s, q_s) - v(s, q_s + 1)]) \right. \right. \\ &\quad \left. \left. - \frac{\gamma}{2} \sigma^2 q_s^2 \right\} ds \right]. \end{aligned}$$

Now, divide by h and let $h \downarrow 0$. Assume also that we can interchange the limit with the supremum and then with the expectation. Moreover, since $(q_s)_{s \in [t, T]}$ is right-continuous, we have that $q_s = q_{s-} = q_t = q$ a.s. for $t \leq s \leq t + h$ and h small enough. Hence,

$$\begin{aligned}
0 &= \sup_{\delta^a, \delta^b \in \mathbb{R}_+} \left\{ v'(t, q) + \mathbb{1}_{\{q > -Q\}} \Lambda^a(\delta^a) (\delta^a - [v(t, q) - v(t, q - 1)]) \right. \\
&\quad \left. + \mathbb{1}_{\{q < Q\}} \Lambda^b(\delta^b) (\delta^b - [v(t, q) - v(t, q + 1)]) - \frac{\gamma}{2} \sigma^2 q^2 \right\} \\
&= v'(t, q) - \frac{\gamma}{2} \sigma^2 q^2 \\
&\quad + \mathbb{1}_{\{q > -Q\}} \sup_{\delta^a \in \mathbb{R}_+} \left\{ \Lambda^a(\delta^a) (\delta^a - [v(t, q) - v(t, q - 1)]) \right\} \\
&\quad + \mathbb{1}_{\{q < Q\}} \sup_{\delta^b \in \mathbb{R}_+} \left\{ \Lambda^b(\delta^b) (\delta^b - [v(t, q) - v(t, q + 1)]) \right\}.
\end{aligned}$$

We define the functions $H^a, H^b: \mathbb{R} \rightarrow \mathbb{R}$ exactly as in the discrete-time model:

$$H^a(p) := \max_{\delta \geq 0} \{(\delta - p) \Lambda^a(\delta)\}, \quad H^b(p) := \max_{\delta \geq 0} \{(\delta - p) \Lambda^b(\delta)\}.$$

We will have more to say about H^a, H^b in the next section. Now we can write

$$\begin{cases} 0 = v'(t, q) - \frac{\gamma}{2} \sigma^2 q^2 + \mathbb{1}_{\{q > -Q\}} H^a(v(t, q) - v(t, q - 1)) \\ \quad + \mathbb{1}_{\{q < Q\}} H^b(v(t, q) - v(t, q + 1)), \\ v(T, q) = 0, \end{cases} \quad (\text{HJB})$$

for all $(t, q) \in [0, T] \times \{-Q, \dots, +Q\}$. This is the HJB equation. Keep in mind that this derivation is only heuristic, since several formal steps were taken without proof. However, under suitable assumptions, one can prove a converse statement: a sufficiently regular solution of the HJB equation coincides with the value function. This is usually called a “verification theorem” in the literature and is the subject of Section 2.5 (see Theorem 2.6).

2.3 Some Mild Assumptions

In order to construct sufficiently regular solutions to the HJB equation and prove the verification theorem, we impose a few additional assumptions.

Recall that $\Lambda^a, \Lambda^b: \mathbb{R}_+ \rightarrow \mathbb{R}_+$ are continuous decreasing functions satisfying the fast-decay condition (1.3). In particular, Λ^a and Λ^b are bounded and the maxima in the definitions of $H^a(p)$ and $H^b(p)$ are attained for every $p \in \mathbb{R}$. Thus, the set-valued maps

$$\Delta^a(p) := \arg \max_{\delta \geq 0} \{(\delta - p) \Lambda^a(\delta)\}$$

and

$$\Delta^b(p) := \arg \max_{\delta \geq 0} \{(\delta - p)\Lambda^b(\delta)\}$$

take non-empty values as subsets of $\mathbb{R}_+$ for all $p \in \mathbb{R}$.

By the axiom of choice, there exist maps $F^a, F^b: \mathbb{R} \rightarrow \mathbb{R}_+$ such that

$$F^a(p) \in \Delta^a(p) \subset \mathbb{R}_+, \quad F^b(p) \in \Delta^b(p) \subset \mathbb{R}_+,$$

for every $p \in \mathbb{R}$,

We now need a very reasonable assumption:

Assumption 2.3. *There exist such functions $F^a, F^b: \mathbb{R} \rightarrow \mathbb{R}_+$ which are also measurable and locally bounded, i.e., bounded on every compact interval.*

Now we turn to the functions H^a, H^b . It is not hard to see that these are non-negative and non-increasing, but they also have sub-linear growth, i.e., there exist $C^a, C^b > 0$ such that, for all $p \in \mathbb{R}$,

$$0 \leq H^a(p) \leq C^a(1 + |p|), \quad 0 \leq H^b(p) \leq C^b(1 + |p|),$$

Indeed, define the functions $g^a(\delta) := \delta\Lambda^a(\delta)$ and $g^b(\delta) := \delta\Lambda^b(\delta)$ on $\mathbb{R}_+$, which are bounded because they are continuous and tend to zero as $\delta \rightarrow +\infty$. Define also

$$C^a := \max\{\|\Lambda^a\|_\infty, \|g^a\|_\infty\}, \quad C^b := \max\{\|\Lambda^b\|_\infty, \|g^b\|_\infty\}.$$

Then, given $p \in \mathbb{R}$, we have that for all $\delta \geq 0$,

$$(\delta - p)\Lambda^a(\delta) \leq g^a(\delta) - p\Lambda^a(\delta) \leq \|g^a\|_\infty + \|\Lambda^a\|_\infty|p| \leq C^a(1 + |p|)$$

and taking the supremum we get the upper bound. The same argument works for H^b . However, we need a bit more than this: at this level of generality we cannot guarantee any regularity on H^a and H^b .

Assumption 2.4. *The functions H^a and H^b are locally Lipschitz.*

2.4 Existence and Uniqueness for the HJB equation

We continue to view the value function as a map $v: [0, T] \rightarrow \mathbb{R}^{2Q+1}$. Note that v is a solution to (HJB) if and only if the time-reversed function $y: [0, T] \rightarrow \mathbb{R}^{2Q+1}$, defined as $y(t) = v(T - t)$, is a solution to the following Initial Value Problem (IVP):

$$\begin{cases} y'(t, q) = -\frac{\gamma}{2}\sigma^2 q^2 + \mathbb{1}_{\{q > -Q\}}H^a(y(t, q) - y(t, q - 1)) & \text{(Time-Reversed HJB)} \\ \quad + \mathbb{1}_{\{q < Q\}}H^b(y(t, q) - y(t, q + 1)), \\ y(0, q) = 0, \end{cases}$$

for all $(t, q) \in [0, T] \times \{-Q, \dots, Q\}$. Thus, existence and uniqueness for the terminal-value problem (HJB) are equivalent to existence and uniqueness for the initial-value problem (Time-Reversed HJB).

We may write (Time-Reversed HJB) in vector form as

$$y'(t) = G(y(t)), \quad y(0) = \mathbf{0},$$

where $G: \mathbb{R}^{2Q+1} \rightarrow \mathbb{R}^{2Q+1}$ is the vector field defined componentwise by

$$[G(x_{-Q}, \dots, x_Q)]_q = -\frac{\gamma}{2}\sigma^2 q^2 + \mathbb{1}_{\{q > -Q\}} H^a(x_q - x_{q-1}) + \mathbb{1}_{\{q < Q\}} H^b(x_q - x_{q+1})$$

for every $q \in \{-Q, \dots, Q\}$.

We know that H^a, H^b have sublinear growth, and we further assumed they are locally Lipschitz. Hence, G inherits these two properties. Under these conditions, it is a well-known fact that the IVP (Time-Reversed HJB) has a unique global solution of class $C^1([0, T]; \mathbb{R}^{2Q+1})$. We therefore obtain the following result:

Theorem 2.5. *For all $T > 0$, there exists a unique $w \in C^1([0, T]; \mathbb{R}^{2Q+1})$ solution to (HJB).*

2.5 The Verification Theorem

Theorem 2.6 (Verification Theorem). *Let $w \in C^1([0, T]; \mathbb{R}^{2Q+1})$ be the unique solution to (HJB). Then $w = v$ and the following Markovian controls are optimal:*

$$\delta_t^{a*} = F^a(w(t, q_{t-}) - w(t, q_{t-} - 1)),$$

$$\delta_t^{b*} = F^b(w(t, q_{t-}) - w(t, q_{t-} + 1)).$$

Remark 2.7. The proposed control is indeed admissible. The process $(q_{t-})_t$ is trivially predictable because it is left-continuous and adapted. Moreover, w is continuous, so the process $(w(t, q_{t-}) - w(t, q_{t-} - 1))_t$ is predictable as well, and finally applying the measurable function $F^a: \mathbb{R} \rightarrow \mathbb{R}_+$ gives that $(\delta_t^{a*})_t$ is a non-negative predictable process. On the other hand, w is bounded and F^a is locally bounded, so $(\delta_t^{a*})_t$ is bounded as well. The same arguments apply to $(\delta_t^{b*})_t$.

Remark 2.8. Plugging δ_t^{a*} and δ_t^{b*} we see that the intensities λ_t^a and λ_t^b depend only on q_{t-} for each $t \in [0, T]$. Thus, the inventory process $(q_t)_t$ solves a Markovian jump SDE.³ Again, in this setting, existence and uniqueness of a solution to this Jump SDE is established in Chapter IV of [5]. Well-posedness of this Jump SDE is crucial for the Verification Theorem 2.6, as we explicitly use the solution (both for the statement and the proof).

Proof. Fix $(t, q) \in [0, T] \times \{-Q, \dots, Q\}$. For any admissible controls $(\delta_s^a, \delta_s^b)_{s \in [t, T]}$ starting at t , consider the process $w(s, q_s)$. Applying Itô's formula we get

$$d(w(s, q_s)) = w'(s, q_s) ds + (w(s, q_s + 1) - w(s, q_s)) dN_s^b + (w(s, q_s - 1) - w(s, q_s)) dN_s^a.$$

³In simpler terms, the inventory process is just a continuous-time Markov Chain on $\{-Q, \dots, Q\}$ moving a single step left or right at each time, and with reflective boundary.

Thus,

$$\begin{aligned}
w(T, q_T) &= w(t, q_t) + \int_t^T w'(s, q_s) ds + \int_t^T (w(s, q_s + 1) - w(s, q_s)) dN_s^b \\
&\quad + \int_t^T (w(s, q_s - 1) - w(s, q_s)) dN_s^a, \\
0 &= w(t, q) + \int_t^T w'(s, q_s) ds + \int_t^T (w(s, q_s + 1) - w(s, q_s)) dN_s^b \\
&\quad + \int_t^T (w(s, q_s - 1) - w(s, q_s)) dN_s^a,
\end{aligned}$$

where we used $w(T, \cdot) = 0$ and $q_t = q$ a.s. in the second equality. Taking expectations, the usual martingale trick (note that the integrands are all bounded) allows us to write

$$\begin{aligned}
0 &= w(t, q) + \mathbb{E}^{(t, q)} \left[\int_t^T w'(s, q_s) ds + \int_t^T (w(s, q_s + 1) - w(s, q_s)) dN_s^b \right. \\
&\quad \left. + \int_t^T (w(s, q_s - 1) - w(s, q_s)) dN_s^a \right] \\
0 &= \mathbb{E}^{t, q} \left[\int_t^T \left\{ w'(s, q_s) - \mathbb{1}_{\{q_{s-} > -Q\}} \Lambda^a(\delta_s^a) (w(s, q_s) - w(s, q_s - 1)) \right. \right. \\
&\quad \left. \left. - \mathbb{1}_{\{q_{s-} < Q\}} \Lambda^b(\delta_s^b) (w(s, q_s) - w(s, q_s + 1)) \right\} ds \right].
\end{aligned}$$

Then we may write the objective function as

$$\begin{aligned}
&\mathbb{E}^{t, q} \left[\int_t^T \left\{ \delta_s^a \lambda_s^a + \delta_s^b \lambda_s^b - \frac{\gamma}{2} \sigma^2 q_s^2 \right\} ds \right] + 0 \\
&= w(t, q) + \mathbb{E}^{t, q} \left[\int_t^T \left\{ w'(s, q_s) + \mathbb{1}_{\{q_{s-} > -Q\}} \Lambda^a(\delta_s^a) (\delta_s^a - [w(s, q_s) - w(s, q_s - 1)]) \right. \right. \\
&\quad \left. \left. + \mathbb{1}_{\{q_{s-} < Q\}} \Lambda^b(\delta_s^b) (\delta_s^b - [w(s, q_s) - w(s, q_s + 1)]) - \frac{\gamma}{2} \sigma^2 q_s^2 \right\} ds \right] \\
&\leq w(t, q) + \mathbb{E}^{t, q} \left[\int_t^T \left\{ w'(s, q_s) + \mathbb{1}_{\{q_{s-} > -Q\}} H^a(w(s, q_s) - w(s, q_s - 1)) \right. \right. \\
&\quad \left. \left. + \mathbb{1}_{\{q_{s-} < Q\}} H^b(w(s, q_s) - w(s, q_s + 1)) - \frac{\gamma}{2} \sigma^2 q_s^2 \right\} ds \right] \\
&= w(t, q).
\end{aligned}$$

In the fourth line we used the fact that, almost surely, $q_s = q_{s-}$ a.e. in $s \in [t, T]$, and then the fact that w solves (HJB). Taking supremum over all admissible controls starting at t , we get $v(t, q) \leq w(t, q)$.

Moreover, if we use the controls $(\delta_s^{a*}, \delta_s^{b*})_{s \in [t, T]}$, the inequality in the third line

becomes equality. Indeed, almost surely $q_s = q_{s-}$ a.e. in $s \in [0, T]$ and this implies

$$\begin{aligned}
& \mathbb{1}_{\{q_{s-} > -Q\}} \Lambda^a(\delta_s^{a*}) (\delta_s^{a*} - [w(s, q_s) - w(s, q_s - 1)]) \\
& + \mathbb{1}_{\{q_{s-} < Q\}} \Lambda^b(\delta_s^{b*}) (\delta_s^{b*} - [w(s, q_s) - w(s, q_s + 1)]) \\
& = \mathbb{1}_{\{q_{s-} > -Q\}} \Lambda^a(\delta_s^{a*}) (\delta_s^{a*} - [w(s, q_{s-}) - w(s, q_{s-} - 1)]) \\
& + \mathbb{1}_{\{q_{s-} < Q\}} \Lambda^b(\delta_s^{b*}) (\delta_s^{b*} - [w(s, q_{s-}) - w(s, q_{s-} + 1)]) \\
& = \mathbb{1}_{\{q_{s-} > -Q\}} H^a(w(s, q_{s-}) - w(s, q_{s-} - 1)) \\
& + \mathbb{1}_{\{q_{s-} < Q\}} H^b(w(s, q_{s-}) - w(s, q_{s-} + 1)) \\
& = \mathbb{1}_{\{q_s > -Q\}} H^a(w(s, q_s) - w(s, q_s - 1)) + \mathbb{1}_{\{q_s < Q\}} H^b(w(s, q_s) - w(s, q_s + 1))
\end{aligned}$$

a.e. in $s \in [t, T]$ almost surely, so their time integrals over $[t, T]$ coincide almost surely (and thus in expectation). Therefore the supremum is achieved: the controls $(\delta^{a*}, \delta^{b*})$ are optimal and

$$v(t, q) = w(t, q).$$

□

2.6 Numerical Illustration of the Continuous-Time Optimal Strategy

We now illustrate the continuous-time optimal strategy numerically. Since the qualitative mechanisms are the same as in the discrete-time model (cf. Section 1.4), the purpose of this section is not to repeat the previous analysis in full detail, but rather to show that the same effects persist in continuous time.

Throughout, we consider that the dynamics for the ask and bid are symmetric in the sense that $\Lambda = \Lambda^a = \Lambda^b$ and further denote $H = H^a = H^b$, $\Delta = \Delta^a = \Delta^b$ and $F = F^a = F^b$. We consider again the exponential intensity function

$$\Lambda(x) = ce^{-x},$$

with $c = 2$ chosen for numerical convenience. The optimization problems defining H and Δ can again be solved explicitly. More precisely,

$$H(p) := \max_{\delta \geq 0} \{(\delta - p)\Lambda(\delta)\} = \begin{cases} ce^{-(p+1)}, & \text{if } p \geq -1, \\ -cp, & \text{if } p < -1, \end{cases}$$

and the maximizer is unique, given by

$$\Delta(p) := \operatorname{argmax}_{\delta \geq 0} \{(\delta - p)\Lambda(\delta)\} = \{(p + 1)^+\},$$

so it is only possible to set

$$F(p) = (p + 1)^+.$$

We note that this choice satisfies all of our assumptions: Λ is continuous, decreasing and fast-decaying, H is locally Lipschitz and F is measurable and locally bounded. As a result, once the value function has been computed, the optimal bid and ask quotes can be obtained directly at each time and inventory level.

2.6.1 Consistency Check and Profitability

As a first verification of the implementation, we compare the theoretical value function at the origin with its Monte Carlo estimate under the optimal control. Figure 2.1 shows that the empirical average converges to the theoretical value $v(0,0)$, which provides a numerical consistency check for the scheme.

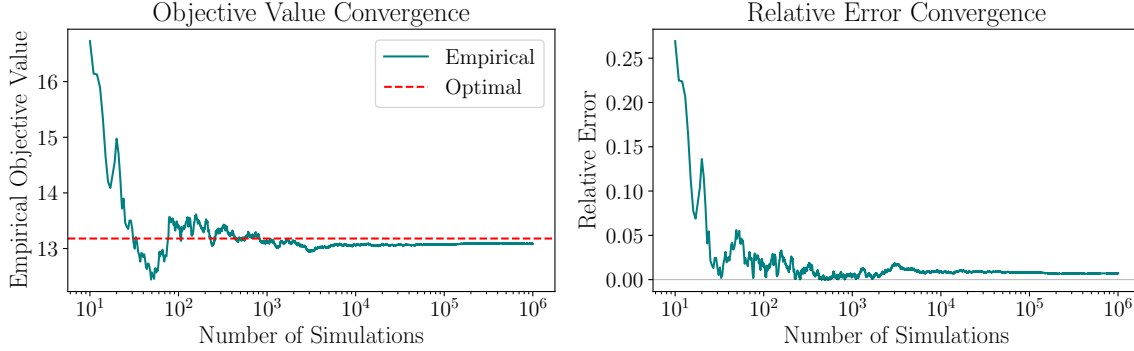


Figure 2.1: Monte Carlo validation of the continuous-time implementation. Left: cumulative average of the simulated objective value together with the theoretical benchmark $v(0,0)$. Right: relative error of the estimator. Time horizon: $T = 10$.

We next turn to the terminal profit and loss. As in the discrete-time setting, we define

$$\text{PnL}_T := X_T + q_T S_T,$$

that is, the sum of the cash position and the marked-to-market value of the terminal inventory. The corresponding distribution is displayed in Figure 2.2. It is approximately Gaussian in shape and centered at a positive value, indicating that the strategy remains profitable on average once inventory risk is properly controlled.

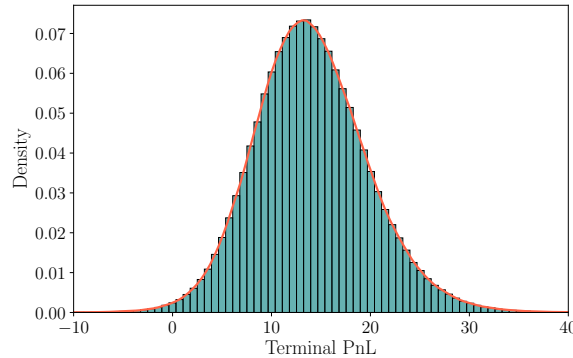


Figure 2.2: Distribution of the terminal PnL generated by the optimal continuous-time strategy over $T = 10$, based on 10^5 simulated trajectories. The distribution is concentrated around a positive mean.

2.6.2 Inventory Control Through Quotes

We now examine how the strategy manages inventory in time. Figure 2.3 displays one simulated price path together with the associated optimal bid and ask quotes,

as well as the resulting inventory process.

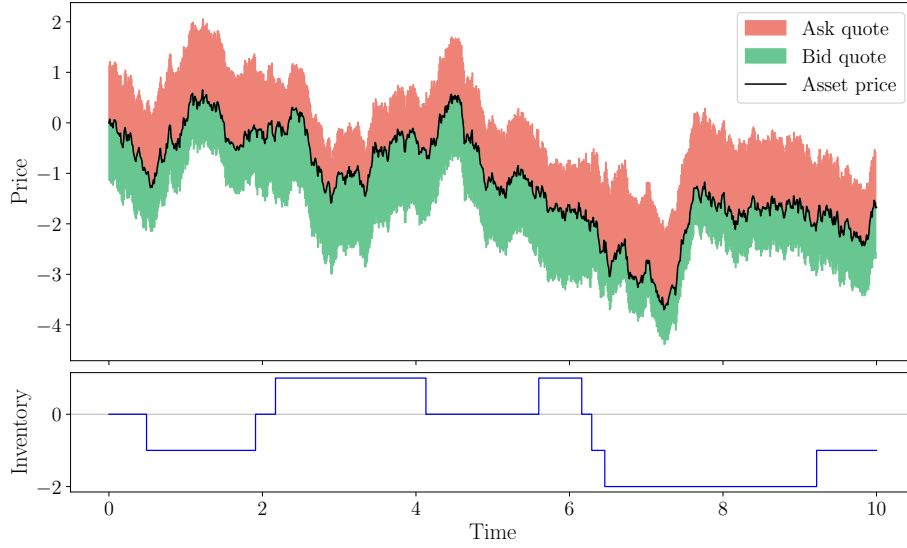


Figure 2.3: Sample path of the asset price with the corresponding optimal bid and ask quotes, together with the inventory process under the optimal strategy.

The same inventory-adjustment mechanism observed in discrete time appears here as well. When the market maker holds a positive inventory, the ask quote is moved closer to the asset price, increasing the probability of selling, while the bid quote is posted farther away, reducing the probability of further purchases. When the inventory is negative, the effect is reversed. In this way, the quotes act as a feedback control whose role is to steer the inventory back toward zero.

This dependence on inventory is shown more clearly in Figure 2.4, where the optimal quotes are plotted against the inventory level at a fixed time. The monotonic structure is exactly the one expected from the theory: ask quotes decrease with inventory, whereas bid quotes increase with inventory.

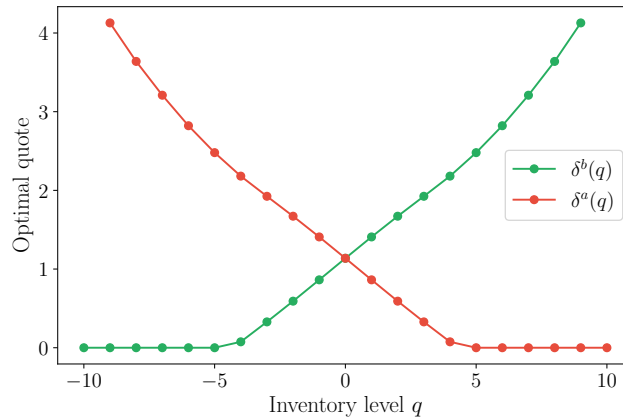


Figure 2.4: Optimal bid and ask quote offsets as functions of the inventory at a fixed time.

2.6.3 Risk–Return Trade-Off

Finally, we investigate how the risk-aversion parameter γ affects performance. As in the discrete-time model, varying γ generates a family of optimal strategies ranging from aggressive, high-risk market making to more conservative inventory control. To summarize this trade-off, Figure 2.5 shows the market maker’s efficient frontier, obtained from 10^5 Monte Carlo simulations for each of the 50 values of $\gamma \in [10^{-4}, 1]$.

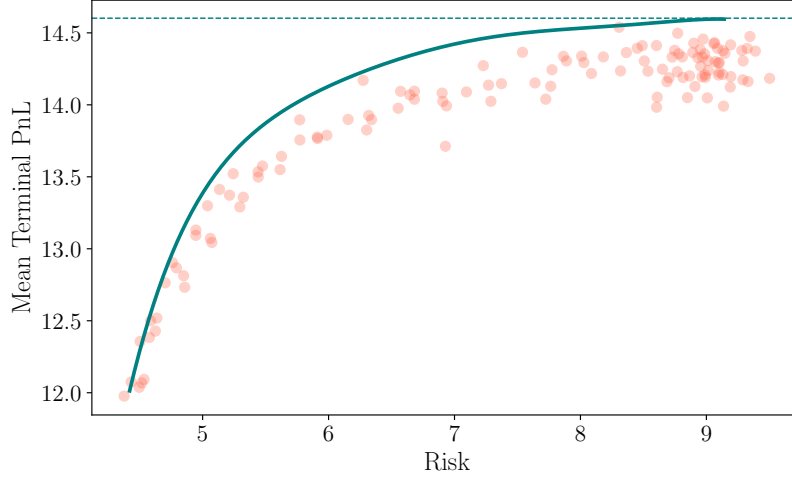


Figure 2.5: Efficient frontier in the continuous-time model: mean terminal PnL as a function of its standard deviation for different values of the risk-aversion parameter and for $T = 10$ fixed. The solid curve corresponds to optimal strategies, while the red points represent perturbed strategies. The dashed line indicates the expected PnL without inventory penalization.

The frontier confirms the same fundamental principle as before: increasing expected profits requires accepting greater dispersion in terminal outcomes.

Appendix A

Simulation Code

All the code developed for this thesis is publicly available on [GitHub](#) (see also [3]). For completeness, the key components of the implementations for both the discrete and the continuous-time models are reproduced below.

A.1 Discrete-Time Model

```
1 import numpy as np
2
3 def dp_step(q, V_qm1, V_q, V_qp1, gamma):
4     """Calculate dynamic programming step value (see Theorem 1.4).
5
6     Args:
7         q (int): Inventory level.
8         V_qm1 (float): Value function at previous inventory level.
9         V_q (float): Value function at current inventory level.
10        V_qp1 (float): Value function at next inventory level.
11        gamma (float): Risk aversion parameter.
12
13    Returns:
14        float: The dynamic programming step value.
15    """
16    step = V_q - gamma / 2.0 * q**2 + H(V_q - V_qm1)
17          + H(V_q - V_qp1)
18    return step
19
20
21
22 def fill_V(T, gamma):
23     """Use dynamic programming to obtain the value function all
24     inventory levels and time steps (see Theorem 1.4).
25
26     Args:
27         T (int): Time horizon.
28         gamma (float): Risk aversion parameter.
29
30    Returns:
31        np.ndarray: Value function array from DP.
```

```

31         Shape (2*T+1, T+1).
32     """
33     V = np.zeros((2*T + 1, T + 1))
34     for n in reversed(range(T)):
35         for q_idx in range(T - n, T + n + 1):
36             q = q_idx - T
37             V[q_idx, n] = dp_step(q,
38                                   V[q_idx - 1, n + 1],
39                                   V[q_idx, n + 1],
40                                   V[q_idx + 1, n + 1],
41                                   gamma
42                                   )
43     return V
44
45
46
47 def run_market_simulation(V, T, sigma, S0, n_sims):
48     """Run several simulations of optimal market making.
49
50     Args:
51         V (np.ndarray): Value function array from dynamic
52                        programming, shape (2*T+1, T+1).
53                        Output from fill_V(T, gamma).
54         T (int): Time horizon.
55         sigma (float): Volatility of the stock price.
56         S0 (float): Initial stock price.
57         n_sims (int): Number of simulations to run.
58
59     Returns:
60         np.ndarray: Terminal PnL for all simulations.
61     """
62     # Initialize arrays for all simulations at once
63     q = np.zeros((n_sims, T + 1), dtype=int)
64     cash = np.zeros((n_sims, T + 1))
65     S = np.zeros((n_sims, T + 1))
66     S[:, 0] = S0
67
68     for n in range(T):
69         # Compute deltas for all simulations
70         q_idx = q[:, n] + T
71         da = Delta(V[q_idx, n + 1] - V[q_idx - 1, n + 1])
72         db = Delta(V[q_idx, n + 1] - V[q_idx + 1, n + 1])
73
74         # Compute probabilities
75         p_ask = f(da)
76         p_bid = f(db)
77         p_none = np.maximum(0.0, 1.0 - p_ask - p_bid)
78
79         # Normalize probabilities to handle numerical errors
80         p_total = p_ask + p_bid + p_none
81         p_ask /= p_total
82         p_bid /= p_total
83         p_none /= p_total

```

```

84
85     # Generate random events for all simulations at once
86     rand_vals = np.random.random(n_sims)
87     eta_a = (rand_vals < p_ask).astype(int)
88     eta_b = ((rand_vals >= p_ask) &
89              (rand_vals < p_ask + p_bid)).astype(int)
90
91     # Update inventory and cash
92     q[:, n + 1] = q[:, n] - eta_a + eta_b
93     cash[:, n + 1] = cash[:, n] + eta_a * (S[:, n] + da) \
94                      - eta_b * (S[:, n] - db)
95
96     # Update stock prices
97     S[:, n + 1] = S[:, n] + np.random.normal(0, sigma, n_sims)
98
99     # Terminal PnL for all simulations
100     pnl_T = cash[:, -1] + q[:, -1] * S[:, -1]
101     return pnl_T

```

A.2 Continuous-Time Model

```

1  import numpy as np
2  from scipy.integrate import solve_ivp
3
4  def solve_V_and_quote_tables(gamma, Q, T, sigma, qs, n_steps,
5                               time_grid, H, F, method="BDF"):
6
7      """Solve the backward ODE for the continuous-time value
8      function and compute optimal quotes as tables.
9
10     Args:
11         gamma (float): Risk aversion parameter.
12         Q (int): Maximum inventory level. The inventory grid is {-Q
13         , ..., Q}.
14         T (float): Time horizon.
15         sigma (float): Volatility of the asset price.
16         qs (np.ndarray): Array of inventory levels.
17         n_steps (int): Number of time steps used in the simulation
18         grid.
19         time_grid (np.ndarray): Forward time grid on [0, T] used
20         for interpolation.
21         H (callable): Hamiltonian function appearing in the ODE.
22         F (callable): Function mapping value-function differences
23         to optimal quote offsets.
24         method (str, optional): Numerical method used by scipy.
25         integrate.solve_ivp.
26
27         Defaults to "BDF".
28
29     Returns:
30         dict: Dictionary containing:
31             - "gamma" (float): Risk aversion parameter.

```

```

23         - "sol" (OdeResult): Raw ODE solution returned by
solve_ivp.
24         - "V_grid" (np.ndarray): Value function interpolated
on the forward
25         simulation grid, shape (n_steps+1, 2*Q+1).
26         - "delta_b_table" (np.ndarray): Optimal bid quote
offsets for each
27         time and inventory level, shape (n_steps+1, 2*Q+1).
28         - "delta_a_table" (np.ndarray): Optimal ask quote
offsets for each
29         time and inventory level, shape (n_steps+1, 2*Q+1).
30     """
31
32     nqs = len(qs)
33     q_to_idx = lambda q: q + Q
34
35     # inventory penalization term
36     def running_term(t, q):
37         return 0.5 * gamma * sigma**2 * q**2
38
39     # ODE for the value function
40     def rhs(t, v):
41         dv = np.zeros_like(v)
42         for i, q in enumerate(qs):
43             val = running_term(t, q)
44             if q > -Q:
45                 val -= H(v[i] - v[i - 1])
46             if q < Q:
47                 val -= H(v[i] - v[i + 1])
48             dv[i] = val
49         return dv
50
51     # backward time grid for ODE solve
52     t_eval_backward = np.linspace(T, 0.0, 2000)
53     vT = np.zeros(nqs)
54
55     # solve ODE
56     sol = solve_ivp(
57         fun=rhs,
58         t_span=(T, 0.0),
59         y0=vT,
60         t_eval=t_eval_backward,
61         method=method,
62     )
63
64     if not sol.success:
65         raise RuntimeError(f"ODE solve failed for gamma={gamma}: {
sol.message}")
66
67     ts = sol.t
68     vs = sol.y
69
70     # interpolate to get a continuous function

```

```

71 V_grid = np.empty((n_steps + 1, nqs))
72 for j, q in enumerate(qs):
73     V_grid[:, j] = np.interp(time_grid, ts[::-1], vs[j, ::-1])
74
75 # compute quote tables
76 delta_b_table = np.full((n_steps + 1, nqs), np.nan)
77 delta_a_table = np.full((n_steps + 1, nqs), np.nan)
78
79 for q in qs:
80     j = q_to_idx(q)
81     if q < Q:
82         delta_b_table[:, j] = F(V_grid[:, j] - V_grid[:, j +
1])
83     if q > -Q:
84         delta_a_table[:, j] = F(V_grid[:, j] - V_grid[:, j -
1])
85
86 return {
87     "gamma": gamma,
88     "sol": sol,
89     "V_grid": V_grid,
90     "delta_b_table": delta_b_table,
91     "delta_a_table": delta_a_table,
92 }
93
94
95
96 def simulate_terminal_pnl_for_gamma(gamma, V_grid, delta_b_table,
97     delta_a_table, q0, Q, T, sigma, S0, x0, dt, n_paths, seed):
98     """Run several simulations of the continuous-time optimal
99     market making strategy.
100
101     Args:
102     gamma (float): Risk aversion parameter.
103     V_grid (np.ndarray): Value function on the simulation grid,
104         shape (n_steps+1, 2*Q+1).
105         Output from solve_V_and_quote_tables
106     (...).
107     delta_b_table (np.ndarray): Optimal bid quote offsets for
108         each time
109         and inventory level, shape (
110         n_steps+1, 2*Q+1).
111     delta_a_table (np.ndarray): Optimal ask quote offsets for
112         each time
113         and inventory level, shape (
114         n_steps+1, 2*Q+1).
115     q0 (int): Initial inventory.
116     Q (int): Maximum inventory level. The inventory is
117     constrained to [-Q, Q].
118     T (float): Time horizon.
119     sigma (float): Volatility of the asset price.
120     S0 (float): Initial asset price.
121     x0 (float): Initial cash account.

```

```

114     dt (float): Time step used for the simulation.
115     n_paths (int): Number of Monte Carlo paths.
116     seed (int): Random seed for reproducibility.
117
118     Returns:
119         dict: Dictionary containing:
120             - "terminal_pnl_paths" (np.ndarray): Terminal PnL for
121               all simulations.
122             - "terminal_payoff_paths" (np.ndarray): Terminal
123               objective value for
124               all simulations, including the running inventory
125               penalty.
126             - "mean_pnl" (float): Mean terminal PnL across all
127               simulations.
128             - "var_pnl" (float): Variance of terminal PnL across
129               all simulations.
130             - "std_pnl" (float): Standard deviation of terminal
131               PnL across all simulations.
132             - "mean_payoff" (float): Mean terminal objective
133               value across all simulations.
134             - "var_payoff" (float): Variance of terminal
135               objective value across all simulations.
136     """
137
138     rng = np.random.default_rng(seed)
139
140     n_steps = int(T / dt)
141     q_to_idx = lambda q: q + Q
142
143     S = np.full(n_paths, S0, dtype=np.float64)      # asset price
144     q = np.full(n_paths, q0, dtype=np.int32)         # inventory
145     x = np.full(n_paths, x0, dtype=np.float64)       # cash account
146
147     # running approximation of integral  $\int_0^T q_t^2 dt$ 
148     inventory_penalty_integral = np.zeros(n_paths, dtype=np.float64)
149
150     sqrt_dt = np.sqrt(dt)
151
152     for k in range(n_steps):
153         inventory_penalty_integral += q**2 * dt
154
155         q_idx = q_to_idx(q)
156
157         # get optimal quotes from tables
158         delta_b = delta_b_table[k, q_idx]
159         delta_a = delta_a_table[k, q_idx]
160
161         # compute Poisson intensities for each path
162         lambda_b = np.where(q < Q, Lambda(delta_b), 0.0)
163         lambda_a = np.where(q > -Q, Lambda(delta_a), 0.0)
164
165         # Poisson draws, capped at 1 per side

```



```

158     bid_exec = np.minimum(rng.poisson(lambda_b * dt), 1)
159     ask_exec = np.minimum(rng.poisson(lambda_a * dt), 1)
160
161     # forbid simultaneous executions
162     both_exec = (bid_exec == 1) & (ask_exec == 1)
163     if np.any(both_exec):
164         # randomly keep only one of the two executions for each
165         # affected path
166         keep_bid = rng.random(size=np.sum(both_exec)) < 0.5
167         bid_exec[both_exec] = keep_bid.astype(int)
168         ask_exec[both_exec] = (~keep_bid).astype(int)
169
170     # update cash / inventory
171     x -= bid_exec * (S - np.nan_to_num(delta_b, nan=0.0))
172     q += bid_exec
173
174     x += ask_exec * (S + np.nan_to_num(delta_a, nan=0.0))
175     q -= ask_exec
176
177     q = np.clip(q, -Q, Q)
178
179     # asset price evolution
180     S += sigma * sqrt_dt * rng.normal(size=n_paths)
181
182     # compute terminal PnL
183     terminal_pnl = x + q * S
184     # associated value for each path
185     terminal_payoff = terminal_pnl - 0.5 * gamma * sigma**2 *
186     inventory_penalty_integral
187
188     return {
189         "terminal_pnl_paths": terminal_pnl,
190         "terminal_payoff_paths": terminal_payoff,
191         "mean_pnl": terminal_pnl.mean(),
192         "var_pnl": terminal_pnl.var(),
193         "std_pnl": terminal_pnl.std(ddof=1),
194         "mean_payoff": terminal_payoff.mean(),
195         "var_payoff": terminal_payoff.var(),
196     }

```

Bibliography

- [1] Marco Avellaneda and Sasha Stoikov. High frequency trading in a limit order book. *Quantitative Finance*, 8:217–224, 04 2008.
- [2] Olivier Guéant. Optimal market making. *SIAM Journal on Financial Mathematics*, 8(1):532–559, 2017. arXiv:1605.01862.
- [3] Carlos Cosentino Corzo, Marçal Herraiz Bayó, and Tiziano Di Gregorio Botek. Market making and optimal inventory management. <https://github.com/TDGB02/Market-Making-and-Optimal-Inventory-Management>, 2026.
- [4] Alexander Barzykin, Philippe Bergault, and Olivier Guéant. Market making by an fx dealer: tiers, pricing ladders and hedging rates for optimal risk control, 2023.
- [5] N. Ikeda and S. Watanabe. *Stochastic Differential Equations and Diffusion Processes*. North-Holland Mathematical Library. North Holland, 2014.